MASTER PROJECT

eXRercise sub-project of the MASTER first Open Call (OC)

eXercise Documentation

NOESISHUB & 8 BELLS

July 28, 2025

Table of Contents

eXRercise sub-project of the MASTER first Open Call (OC)

The following documentation addresses the integration of assets developed using Unity Engine and the Unity Editor, for the purposes of the MASTER OC project **eXercise**.

One of the components of the **eXercise Training System** is the creation of VR-based scenarios. The library described below enables dynamic VR scenarios for: - fire incidents (by creating a virtual fire effect at a robotic arm/device) - malfunction scenarios at a specified robotic arm or device

This component accepts structured decisions output from an LLM prompt, used by a trainer. The result is a VR scenario presented to a trainee. It is distributed as a Unity-native DLL library.

Pilot Project Report: NLP-based Trainer

1 Objective

The primary aim of this pilot project was to create a streamlined, interactive interface enabling trainers to input high-level textual descriptions (prompts) regarding training scenarios involving virtual fires and robotic arm malfunctions. These inputs are processed by advanced language models (LLMs) to generate scenario recommendations, identify malfunctions, and support training decisions.

2 Overview of System

The system has been developed using Streamlit and comprises two key interface sections:

1. Robotic Arm Selector Tab

2. AI Advisor Tab

2.1 Robotic Arm Selector Tab

This tab provides users (trainers) with the ability to:

- Select an LLM model, supporting both OpenAI and Hugging Face APIs (e.g., GPT-4o, Zephyr, Mistral).
- Dynamically input the number of robotic arms for the session.
- Submit textual prompts describing training scenarios.
- Enter distance values for each robotic arm relative to the trainee.

The system then:

- Processes the prompt and distances using the selected LLM.
- Returns structured JSON output indicating:
  - Which robotic arm may be experiencing a virtual fire.
  - Which arm is potentially malfunctioning.
  - An explanation for the AI's decision.
- Displays example prompts to guide user input.
- Persists all session data for later analysis.

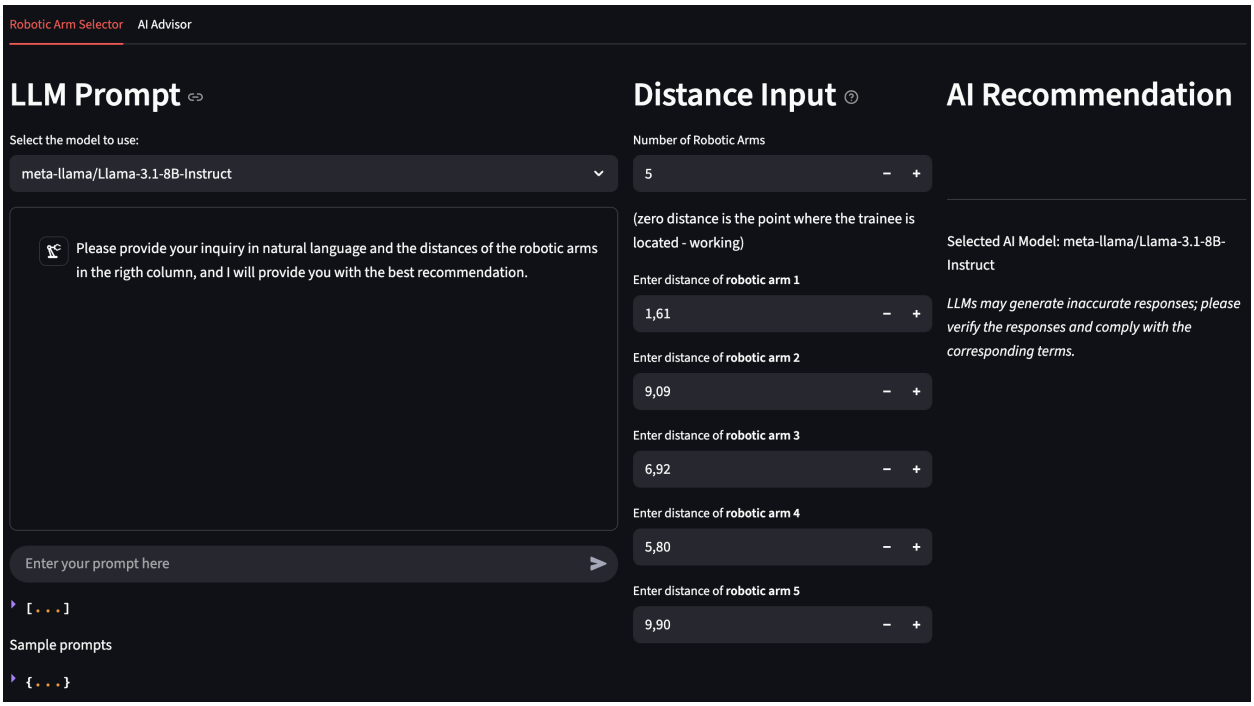


Figure 1: Robotic Arm Selector Tab

2.2 AI Advisor Tab

The **AI Advisor Tab**, available on the eXRercise Hugging Face Interface, supports two advanced functions:

1. **Session Stress Result**
2. **Performance Report Generation**

Functionality Overview

- **Session Stress Results Retrieval:** Connects to an external stress classification API to retrieve emotional/stress performance based on physiological data.
- **Performance Report Generation:** Uses the LLM to generate a markdown report that includes:
  - **Session metadata** (date, time, participant ID)
  - **Stress analysis** results with tables
  - **Recommendations** for future training scenarios

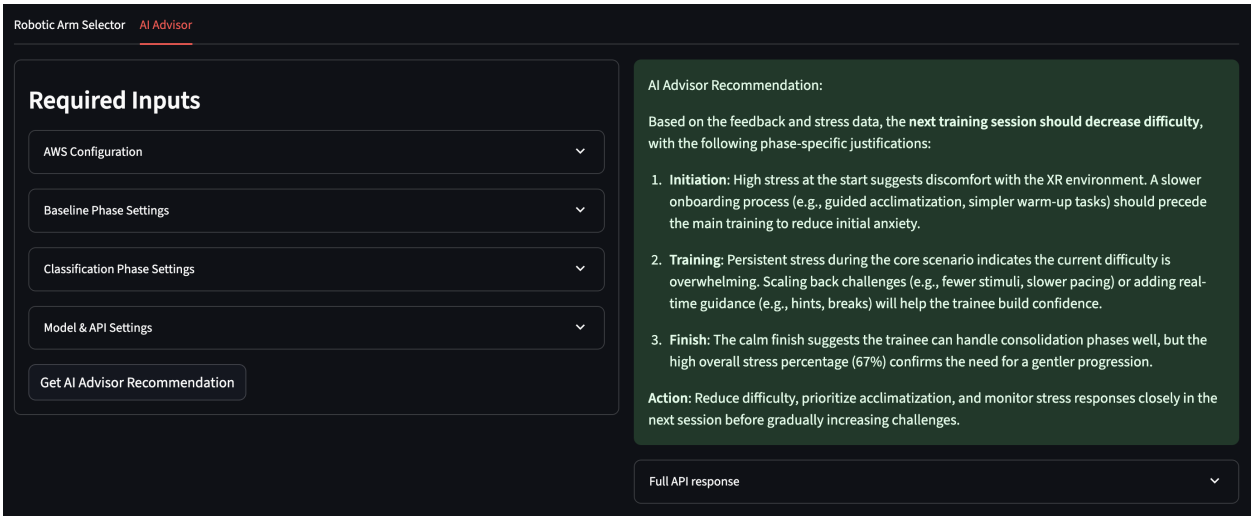


Figure 2: AI Advisor Tab

LLM Compatibility

- Supports **DeepSeek** and **OpenAI's** LLM APIs.

The Session Stress Results Retrieval function currently supports **Empatica Embrace Plus** wristband data from an **AWS S3 Bucket**.

Biometric Signals Collected:

1. Aggregated Per Minute Electrodermal Activity (EDA)
2. Aggregated Per Minute Skin Temperature
3. Accelerometry Standard Deviation

A pre-trained **CNN** performs per-minute stress classification before invoking the LLM.

Required API Parameters

Name	Description
`aws_region_name`	AWS region (e.g., `us-east-1`)
`s3_bucket`	S3 bucket name
`prefix`	Path prefix
`participant_id`	Participant ID
`timezone`	EU timezone (e.g., `Europe/Athens`)
`baseline_date`	Baseline date (YYYY-MM-DD)
`baseline_start_time`	Start time for baseline (HH or HH:MM)
`baseline_end_time`	End time for baseline (HH or HH:MM)
`classification_date`	Classification date (YYYY-MM-DD)
`classification_start_time`	Start time for classification (HH or HH:MM)

Name	Description
`classification_end_time`	End time for classification (HH or HH:MM)
`initiation_phase_duration_min`	Duration in minutes for initiation
`finish_line_phase_duration_min`	Duration in minutes for finish line
`llm_api_key`	(Optional) LLM API key
`llm_model`	(Optional) LLM model (default: `deepseek-chat`)

---

3 System Components

3.1 Natural Language Interface

The platform leverages large language models (via OpenAI and Hugging Face) for prompt processing. The system dynamically constructs a structured prompt containing:

- Trainer's textual input
- Robotic arm distance values
- Request for identification of robotic arm associated with virtual fire and malfunction

3.2 Distance Input Panel

Users can input precise numerical distances for each robotic arm. These values are used by the LLMs to make scenario inferences.

3.3 Model Integration

Models supported include:

- GPT-4o, GPT-4o-mini (OpenAI API, currently disabled options)
- Llama-3.1-8B-Instruct, Mixtral-8x22B-Instruct-v0.1, DeepSeek-V3 (Hugging Face Inference API)

The list of models can be extended to include others.

3.4 Stress Analysis

The system queries a stress analysis API to retrieve a participant's classification result for the session. It matches timestamps and participant IDs to fetch the correct entry.

4 Sample Output Format


```
{
  "Robotic_Arm_Virtual_Fire": 1,
  "Robotic_Arm_Malfunctioning": 2,
  "Reason_of_selection": "Robotic Arm Number 1 is the closest to the base with a distance of 1.43, whic
}
```


Example AI Advisor LLM Response

> "Based on the feedback and stress data, the next training session should decrease difficulty, with the following phase-specific justifications:

Initiation Phase: High stress at the start suggests discomfort with the XR environment. A slower onboarding process (e.g., guided acclimatization, simpler warm-up tasks) should precede the main training to reduce initial anxiety.

Training Phase: Persistent stress during the core scenario indicates the current difficulty is overwhelming. Scaling back challenges (e.g., fewer stimuli, slower pacing) or adding real-time guidance (e.g., hints, breaks) will help the trainee build confidence.

Finish Line Phase: The calm finish suggests the trainee can handle consolidation phases well, but the high overall stress percentage (67%) confirms the need for a gentler progression.

Action: Reduce difficulty, prioritize acclimatization, and monitor stress responses closely in the next session before gradually increasing challenges."

5 Usability and User Experience

- The interface is intuitive and provides real-time feedback.
- Prompts with fewer than 20 characters are disallowed to prevent vague input.
- Prompt inputs are stored and reused for report generation.
- Users can view AI model outputs in both JSON and Chat UI formats.

6 Challenges and Recommendations

- Challenge: Occasional JSON parsing failures from LLM responses.
  Recommendation: Implement retry logic and more constrained prompt formatting.
- Challenge: Ambiguity in prompt interpretation when language is imprecise.
  Recommendation: Enhance prompt scaffolding and allow iterative refinement.

7 Conclusion

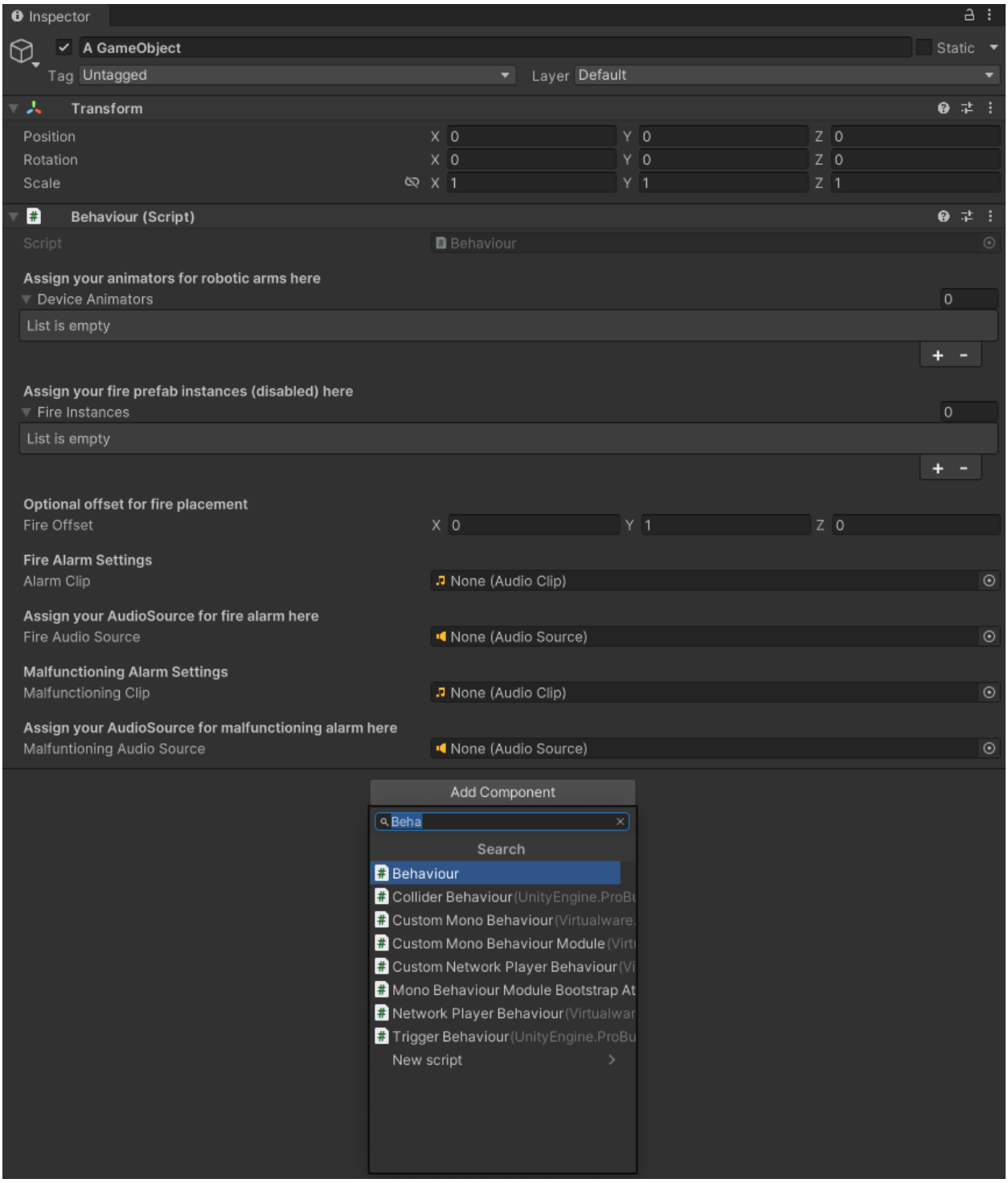
The pilot project successfully demonstrated the feasibility of translating natural language trainer inputs into actionable XR training scenarios. The integration of multiple LLMs, combined with real-time data capture and post-session reporting, creates a powerful tool for enhancing safety training simulations.

XR Scenario Creator

1 Components Implemented

Two key components have been developed: - **BusinessLogic** component - **HuggingFaceJsonFetcher** component

The `BusinessLogic` is designed to accommodate the virtual robotic arms used in your Unity scene. You can assign it via the Unity Inspector, as shown below:



Example showing the BusinessLogic component fields in the Unity Inspector.

---

2 BusinessLogic Component Fields Overview

All fields listed below are customizable for flexibility across different environments and training goals.

Field Name	Type	Description
`deviceAnimators`	Animator[]	Robotic Devices with Animators attached
`fireInstances`	GameObject[]	Prefabs for fire effects
`fireOffset`	Vector3	Offset to place fire effects
`alarmClip`	AudioClip	Sound to play during fire incident
`fireAudioSource`	AudioSource	Audio source for fire alarms
`malfunctioningClip`	AudioClip	Sound to play during malfunction incident
`malfuntioningAudioSource`	AudioSource	Audio source for malfunction alerts

2.1 BusinessLogic Component Field Descriptions

2.1.1 `deviceAnimators` (Animator[]) A list of Animator components. Assign any GameObject (e.g., robotic arms) that contains an Animator. If no Animator is present, it will not work.

2.1.2 `fireInstances` (GameObject[]) A pool of disabled fire prefabs placed in the scene. The script will activate one randomly during a fire event.

2.1.3 `fireOffset` (Vector3) Offset where the fire prefab appears relative to the device. Default is `(0, 1, 0)`.

2.1.4 `alarmClip` (AudioClip) The sound that plays during a fire incident. Use a looping alarm or suitable alert.

2.1.5 `fireAudioSource` (AudioSource) Audio source used to play `alarmClip`.

2.1.6 `malfunctioningClip` (AudioClip) Audio clip triggered when a malfunction event occurs.

2.1.7 `malfuntioningAudioSource` (AudioSource) Audio source used to play the `malfunctioningClip`.

---

3 Unity Integration Guide – HuggingFace JSON Fetcher

This section explains how to use the `HuggingFaceJsonFetcher` Unity component alongside `BusinessLogic` to create automated, LLM-driven VR scenarios using structured JSON input.

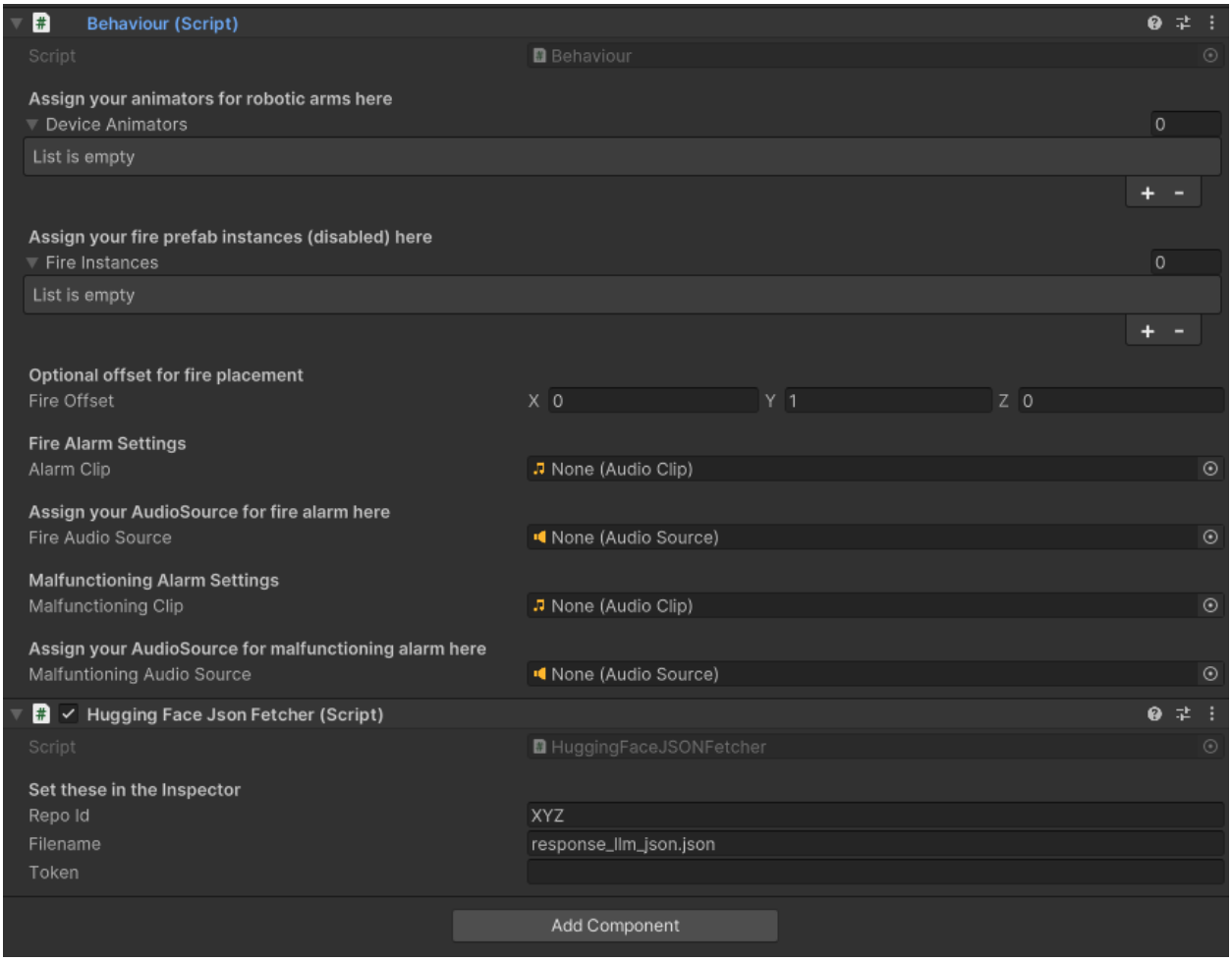
The trainer promets in the provided LLM, the desired scenario (fire, malfuncationing incident or even both) and the LLM return a selection of the selected robotic arm which will virtually be set on fire. When the traineed start the VR application, the library retrieves the structured ouput of the LLM, which indicated the robotic devices to be set on fire, malfunction or in case of a combination (a selection of max. 2 robotic devices) of the incidents which coresponding robotic device suffers which disaster.

An example structured output is shown here, presenting also the JSON structure of the ouput:

```
{
  "Robotic_Arm_Virtual_Fire": 1,
  "Robotic_Arm_Malfunctioning": 2,
  "Reason_of_selection": "Robotic Arm Number 1 is the closest to the base with a distance of 1.43, whic
}
```

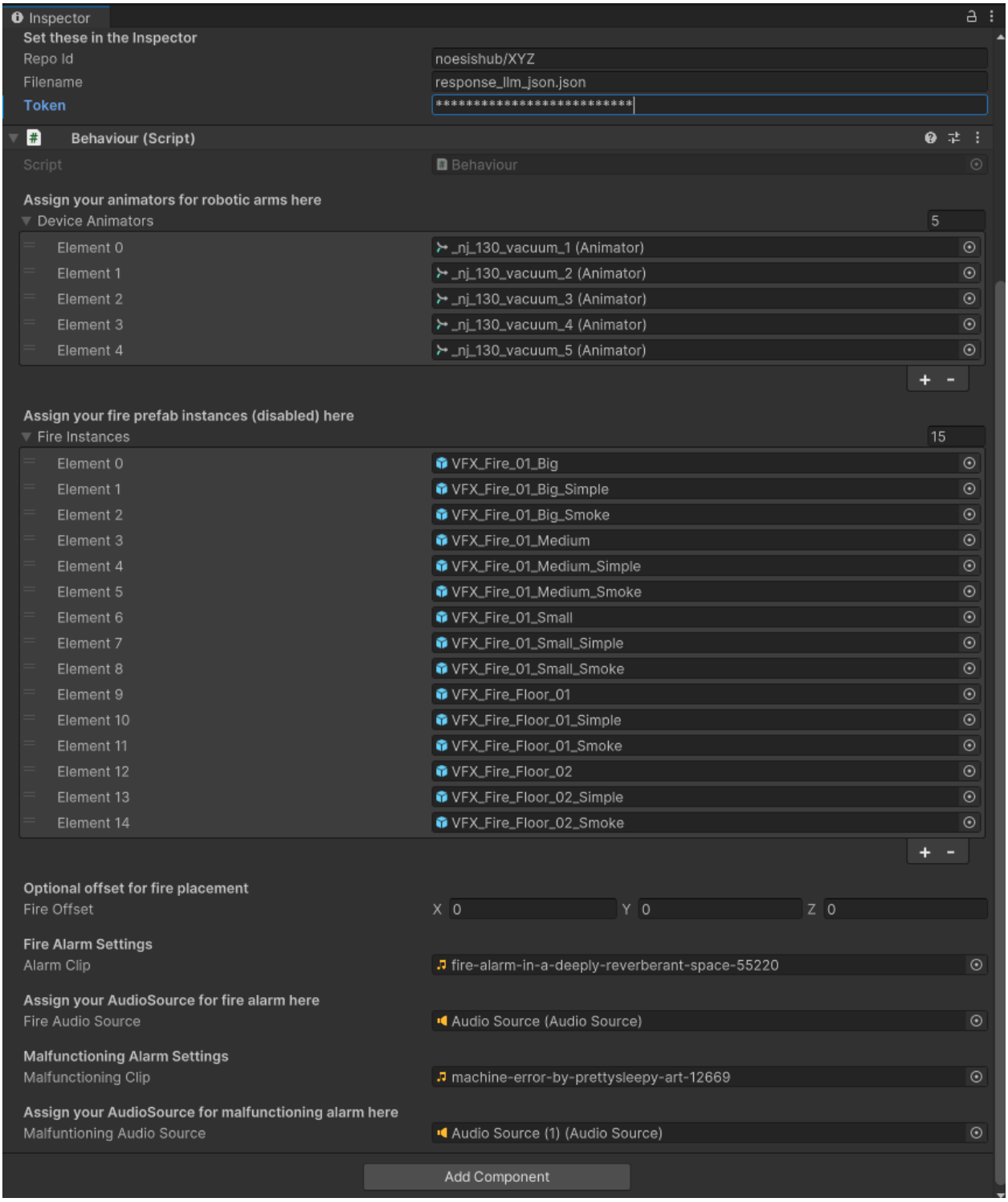
This component, retrieves the LLM structured output from a dataset repository, which must be created in HuggingFace (see the corresponding documentation). In order to successfuly retrieve this information, we have to correctly set the following fields, show in the table below:

Parameter	Type	Description
repoId	string	Hugging Face dataset repository ID (e.g., noesishub/XYZ).
filename	string	JSON file name inside the dataset.
token	string	Hugging Face access token. Required for private datasets (please contact us at **info@noesishub.com** to provide you with a testing token).



Example showing the HuggingFaceJsonFetcher component fields in the Unity Inspector.

Below we show how a fully configured BusinessLogic and HuggingFaceJsonFetcher component looks like.



Example showing how both components can be configured in the Unity Inspector. This is a fully functional configuration.

3.1 HuggingFaceJsonFetcher Field Descriptions

3.1.1 `repoId` (string) The repoId is the name of the created dataset in Hugging Face (this is also the name that must be used in the LLM module in HugginFace). No links or any url of any kind is needed, only

the profile name followed with the dataset name e.g. my-user-name/dataset-name. Inside this repository there must be at least one file (default can be the file with name response_llm_json.json) in which the LLM will write its structured output.

3.1.2 `filename` (string) The name of the file the LLM writes the structured output in the designated dataset (see above field).

3.1.3 `token` (string) In order to access the Hugging Face dataset, we need to create a token from the Hugging Face user account in order for the library to be authenticated and authorized to access the file mentioned above (same logic as in github, gitlab, etc). Refer to this link for additional information on this topic. Be aware to set the correct priviledges, otherwise the library will now be able to access the file, resulting in a 401 error response.

> **BE CAREFULL not to publish or accidentally share the generated token. Line other tokens, is must not be publically available!**

4 Usage Hints & Tips

Here are some hints and tips for taking care of some non-automated settings and options. We are considering to integrate these as automatic mechanisms to pre-emptivelly manage and handle setup mistakes.

- The number of the RoboticArms inside the Unity application must be the same as in the Robotic Arm Selector UI as in the Number of Robotic Arms value set there.
- Reading the LLM structured output, which is written inside a hugging face dataset, required a token. The dataset name is noesishub/XYZ, and the token can be provided by contacting us via the following mail: **info@noesishub.com**

eXRercise Training System: Integration Guide

Welcome to the documentation for the **eXRercise Training System**, a pilot developed under the MASTER OC initiative. This guide walks you through how to:

- Set up the Unity DLL-based VR scenario system
- Connect and control it using a Streamlit UI with LLMs
- Deploy integrated XR training simulations with biometric feedback

This documentation assumes you know your way around Unity, Python, and Hugging Face/Streamlit.

---

1. Setting Up the Unity Plugin

Overview

The Unity module delivers interactive XR scenarios using robotic arms. It uses two core components:

- `BusinessLogic`: Controls animations, fires, and malfunctions.
- `HuggingFaceJsonFetcher`: Reads JSON scenario data from Hugging Face.

1.1 Import the DLL

1. Copy the distributed `.dll` file into `Assets/Plugins` in Unity.
2. Attach the `BusinessLogic` and `HuggingFaceJsonFetcher` scripts to a GameObject in your scene.

1.2 Configure BusinessLogic

Use the Unity Inspector to set the following:

Field	Description
`deviceAnimators`	Animator components for each robotic arm
`fireInstances`	Pool of fire prefabs
`fireOffset`	Offset vector (default: 0,1,0)
`alarmClip`	Audio for fire events
`fireAudioSource`	Source that plays fire audio
`malfunctioningClip`	Audio for malfunctions
`malfuntioningAudioSource`	Source for malfunction audio

1.3 Configure HuggingFaceJsonFetcher

Set the following fields in the Inspector:

Field	Description
`repoId`	e.g., `noesishub/XYZ`
`filename`	JSON file (e.g., `response_llm_json.json`)
`token`	Hugging Face API token

1.4 Sample JSON Format

```
{
  "Robotic_Arm_Virtual_Fire": 1,
  "Robotic_Arm_Malfunctioning": 2,
  "Reason_of_selection": "some explanation here."
}
```

The `BusinessLogic` component reads this JSON and dynamically triggers visual and audio events accordingly.

---

2. Streamlit UI Setup for Trainer Control

Overview

This interface allows trainers to define XR scenarios using natural language prompts and control Unity behavior indirectly via JSON files written to Hugging Face.

2.1 Interface Structure

The app contains:

- **Robotic Arm Selector Tab**: Prompt-based scenario generator
- **AI Advisor Tab**: Performance and stress analysis

2.2 Robotic Arm Selector

Functionality includes:

- Prompt input (e.g., “Set fire to closest robotic arm”)
- Distance data for robotic arms

- LLM selection (OpenAI / Hugging Face)
- Output: JSON file uploaded to Hugging Face

  The prompt must describe with at least 20 characters the desired scenario. The trainer can either explicitely ask for only a fire or only a malfunctioning effect, or a combination.

Sample Output:

```
{
  "Robotic_Arm_Virtual_Fire": 1,
  "Robotic_Arm_Malfunctioning": 3,
  "Reason_of_selection": "the explanation here for the selection"
}
```

  This file must match the Unity-side `filename` in HuggingFaceJsonFetcher!

  The numeric values can range from 1 to the actual number of the robotic arms. If the value is set to -1, no arm is selected by the LLM.

2.3 AI Advisor

Provides advanced analytics:

- Retrieves biometric stress data (from Empatica devices)
- Classifies session difficulty using CNN models
- Generates markdown reports via LLMs

2.4 Required Configuration

Param	Description
`aws_region_name`	AWS region for S3
`s3_bucket`	Data source bucket
`participant_id`	Identifier
`baseline_date`, `start_time`, etc.	Time frame for analysis

2.5 Notes

- Prompts shorter than 20 characters are disallowed.
- The system supports retry logic on JSON parsing failures.

---

3. Conclusion

The eXRercise system demonstrates the integration of Unity VR capabilities with LLM-driven training scenarios and biometric analysis. Follow the guide above to deploy and test your own scenarios using this hybrid XR + AI training platform.